SPRING SECURITY

Resource Server Integration Testing Lab

Progressive challenges with and without MockMvc

GOAL

Write integration tests for the existing Order Resource Server. You will test endpoint authorization, method security, real UserDetails lookup, JWT-based authentication, and full HTTP behavior using several Spring Security test techniques.

Techniques practiced

- @SpringBootTest with direct service invocation - no MockMvc
- @WithMockUser and @WithUserDetails
- @SpringBootTest + @AutoConfigureMockMvc
- MockMvc requests authenticated with jwt()
- MockMvc requests that exercise the real JwtDecoder and authority converter
- @SpringBootTest(webEnvironment = RANDOM_PORT) without MockMvc

Important testing rule

@WithMockUser and @WithUserDetails populate the SecurityContext of the test thread. They are excellent for direct secured-method tests and MockMvc-based tests. They do not authenticate a real HTTP request sent to a server running on a different thread; for that case, put a bearer token on the HTTP request.

Application under test

Use the same Order Resource Server from the previous exercises. Authentication is JWT-based, endpoint authorization is configured in the SecurityFilterChain, and sensitive business operations are also protected with method security.

Method	Endpoint	Expected authorization	Layer(s)
GET	/api/products	Public	HTTP + method where applicable
POST	/api/orders	Authenticated user	HTTP + method where applicable
GET	/api/orders/{id}	Owner or ADMIN	HTTP + method where applicable
DELETE	/api/orders/{id}	Authenticated; service rules still apply	HTTP + method where applicable
PUT	/api/orders/{id}/cancel	MANAGER or ADMIN	HTTP + method where applicable
PUT	/api/orders/{id}/refund	ORDER_REFUND and SCOPE_orders.refund	HTTP + method where applicable
PUT	/api/orders/{id}/status	MANAGER/ADMIN according to status rule	HTTP + method where applicable
GET	/api/admin/reports	ADMIN	HTTP + method where applicable

Known users

- alice - CUSTOMER; owns order 1 and order 3
- bob - CUSTOMER + ORDER_REFUND; owns order 2
- manager - MANAGER
- admin - ADMIN

Lab rule
Do not disable the Spring Security filter chain in these tests. The purpose of the lab is to exercise security configuration, not merely controller behavior.

Challenge 1 | Method security with @WithMockUser - no MockMvc

Start at the service layer. Load the real Spring application context and call a secured service method directly.

```
@SpringBootTest
class OrderServiceSecurityIT {

    @Autowired
    OrderService orderService;

    // TODO tests
}
```

Your task

- Create an integration test class annotated with @SpringBootTest.
- Inject the real OrderService bean.
- Write one test as a user with role MANAGER and verify that cancelAnyOrder(...) succeeds.
- Write one test as a CUSTOMER and verify that invoking the same method is rejected.
- Assert the security failure explicitly instead of accepting any exception.

Verify

- The positive test really changes the order state.
- The negative test proves the method body did not perform the forbidden operation.

How to solve it

Use @WithMockUser to populate the test SecurityContext. Because the test calls the Spring-proxied service bean on the same thread, method security sees the mocked Authentication. For the negative case, assert AuthorizationDeniedException (or the exception type used by your Spring Security version). This test verifies the @PreAuthorize rule and Spring method-security proxy, but it does not test HTTP request matching or JWT decoding.

Challenge 2 | Use the real UserDetailsService with @WithUserDetails

Repeat a method-security integration test, but authenticate from an actual user stored by the application instead of inventing authorities in the annotation.

Your task

- Use @SpringBootTest and inject the secured service bean.
- Write a test annotated with @WithUserDetails("manager") and verify a manager-only operation succeeds.
- Write another test with @WithUserDetails("alice") and verify it is denied.
- Inspect the Authentication principal in one test and verify it came from the application UserDetailsService.

Verify

- Use deterministic test data so manager and alice always exist.
- Do not duplicate the roles in the test annotation - the point is to retrieve them.

How to solve it

With @WithUserDetails, Spring Security calls the configured UserDetailsService and builds the test Authentication from the returned UserDetails. This is useful when your application has custom principals or when you want the test to depend on the real stored authorities. Unlike @WithMockUser, the username must actually exist in the test data.

Challenge 3 | Endpoint rules with MockMvc + @WithMockUser

Move to the web layer while still loading the complete application context.

```
@SpringBootTest
@AutoConfigureMockMvc
class OrderEndpointSecurityIT {

    @Autowired
    MockMvc mvc;

    // TODO tests
}
```

Your task

- Annotate the class with @SpringBootTest and @AutoConfigureMockMvc.
- Inject MockMvc.
- Verify GET /api/products succeeds without authentication.
- Verify GET /api/admin/reports is forbidden to a CUSTOMER.
- Verify GET /api/admin/reports succeeds with @WithMockUser(roles = "ADMIN").
- Verify an authenticated order request reaches the controller/service instead of being rejected by the filter chain.

How to solve it

MockMvc executes the real Spring MVC pipeline and Spring Security filter chain without starting a servlet container. @WithMockUser sets up the SecurityContext used by the MockMvc request, so request matchers and role checks can be exercised quickly. Remember that the Authentication type is a UsernamePasswordAuthenticationToken, not JwtAuthenticationToken; use this style only for rules that do not require JWT-specific principal/claim behavior.

Challenge 4 | Owner checks with MockMvc + @WithUserDetails

Test an endpoint whose final decision depends on the authenticated username and a real order owner stored in the application.

Your task

- Use the real application context and MockMvc.
- Call GET /api/orders/1 as alice using @WithUserDetails("alice") and expect success.
- Call GET /api/orders/2 as alice and expect 403.
- Call GET /api/orders/2 as admin and expect success.
- Verify that the test exercises the custom OrderSecurity bean rather than stubbing its answer.

How to solve it

Seed known ownership data for the integration test. @WithUserDetails gives the request the same principal shape your application normally creates from its UserDetailsService. MockMvc then traverses the filter chain, controller, service proxy, @PreAuthorize expression, and OrderSecurity bean. This is a useful test for authorization logic that combines request handling with database/repository state.

Challenge 5 | Resource Server authorization with MockMvc jwt()

Now test as an OAuth 2.0 Resource Server user rather than as a username/password Authentication.

```
mvc.perform(put("/api/orders/{id}/refund", 2)
      .with(jwt()
        // TODO principal / claims / authorities
      ))
   .andExpect(status().isOk());
```

Your task

- Write a MockMvc test using .with(jwt()).
- Create a JWT principal for bob and grant ORDER_REFUND plus SCOPE_orders.refund.
- Verify PUT /api/orders/2/refund succeeds.
- Create another JWT with ORDER_REFUND but without SCOPE_orders.refund and verify the request is forbidden.
- Create a JWT with the scope but without ORDER_REFUND and verify it is also forbidden.
- Add an assertion that the response/business state changed only in the successful case.

How to solve it

Use SecurityMockMvcRequestPostProcessors.jwt() to place a mock JwtAuthenticationToken into the request. Supply the authorities needed by the authorization expression. This avoids signing a real token and avoids contacting an Authorization Server, so the test stays focused on the Resource Server authorization rule. Be careful: if you explicitly set authorities on jwt(), you are not proving that your production claim-to-authority converter created them correctly.

Challenge 6 | Test JWT claim-to-authority conversion

Close the gap left by jwt(): prove that your actual Resource Server converter turns token claims into the authorities used by Spring Security.

```
mvc.perform(put("/api/orders/{id}/refund", 2)
      .header(HttpHeaders.AUTHORIZATION, "Bearer test-token"))
   .andExpect(status().isOk());
```

Your task

- Keep @SpringBootTest + @AutoConfigureMockMvc.
- Replace the JwtDecoder bean in the test context with a test/mock implementation.
- Configure the decoder to return a Jwt containing sub=bob, the application authority claim, and scope=orders.refund.
- Send a normal HTTP Authorization header: Bearer test-token.
- Verify the refund request succeeds when the returned Jwt contains the required claims.
- Return a Jwt missing one required claim and verify the request is forbidden.

How to solve it

The bearer-token filter now runs normally. It extracts the header, calls the JwtDecoder, obtains your test Jwt, and passes that Jwt through the application JwtAuthenticationConverter. This is the right integration test when you specifically want to verify claim mapping. Prefer Spring Framework's test bean replacement support (for example @MockitoBean where available) or a dedicated @TestConfiguration with a test JwtDecoder.

Challenge 7 | Full HTTP integration test - no MockMvc

Start a real embedded server and prove that the Resource Server protects requests over an actual HTTP connection.

Your task

- Use @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT).
- Use a real HTTP test client (for example RestTestClient, WebTestClient, RestClient, or TestRestTemplate according to your project setup).
- Verify GET /api/products works anonymously.
- Verify a protected endpoint returns 401 when no bearer token is sent.
- Configure a deterministic test JwtDecoder or a test signing key so a known bearer token can be accepted.
- Send the bearer token in the Authorization header and verify an allowed endpoint returns success.
- Send a token without sufficient authority and verify 403.

Verify

- Include at least one 401 assertion and one 403 assertion.
- Explain in a code comment why @WithMockUser would not authenticate this request.

How to solve it

RANDOM_PORT starts a real server, so authentication must travel with the HTTP request. @WithMockUser and @WithUserDetails do not authenticate that request because the test and server run on different threads. Use a deterministic test JwtDecoder or a JWT signed by a trusted test key.

Challenge 8 | Capstone - choose the smallest test that proves each rule

Demonstrate that you can select a testing technique intentionally instead of defaulting every scenario to the heaviest setup.

Your task

- Write one direct @WithMockUser service test for a pure @PreAuthorize role rule.
- Write one @WithUserDetails test for authorization that depends on a stored user/principal.
- Write one MockMvc + jwt() test for a scope/authority combination.
- Write one MockMvc + bearer header test that proves the real JWT authority converter.
- Write one RANDOM_PORT test for full HTTP behavior.
- For each test, add a one-line comment: "This test proves ..." and "This test intentionally does not prove ...".

Verify

- Avoid repeating the same rule in five test styles.
- Keep each test deterministic and independent.
- Reset mutable order data between tests if a test changes status.

How to solve it

Think in layers. Direct secured-method tests are fastest for method authorization. MockMvc is ideal for the filter chain + MVC + method security without a server. jwt() is ideal when token cryptography and conversion are irrelevant. A mocked/test JwtDecoder plus a bearer header proves the converter path. RANDOM_PORT is the broadest test and should be used selectively for behavior that specifically needs a real HTTP server.

Completion checklist

- At least two integration tests without MockMvc.
- At least three integration tests using MockMvc.
- Use @WithMockUser at least once.
- Use @WithUserDetails at least once.
- Use jwt() at least once.
- Test both 401 Unauthorized and 403 Forbidden.
- Test at least one method-level authorization rule and one endpoint-level rule.
- Test at least one JWT authority/scope rule.
- Include one test that exercises your real claim-to-authority converter.
- Keep security enabled in every test.

Expected takeaway

Integration testing Spring Security is not one technique. Choose the narrowest test that still proves the security behavior you care about, and know which pieces are mocked or bypassed by each approach.